Repaso de Pandas

PROFESORA VICTORIA GUTIERREZ

Python

Librerías

Son colecciones de funciones y clases predefinidas y reutilizables que facilitan el uso del lenguaje

Librería estándar

No necesito importarlas

Tipos de datos integrados: int, float, booleanos, string, etc

Procesamiento de texto

Procesamiento de fechas

Funciones Matemáticas

Etc, etc, etc

Librerías de terceros

Necesito importarlas

NumPy

matplotlib

pandas

scikit learn

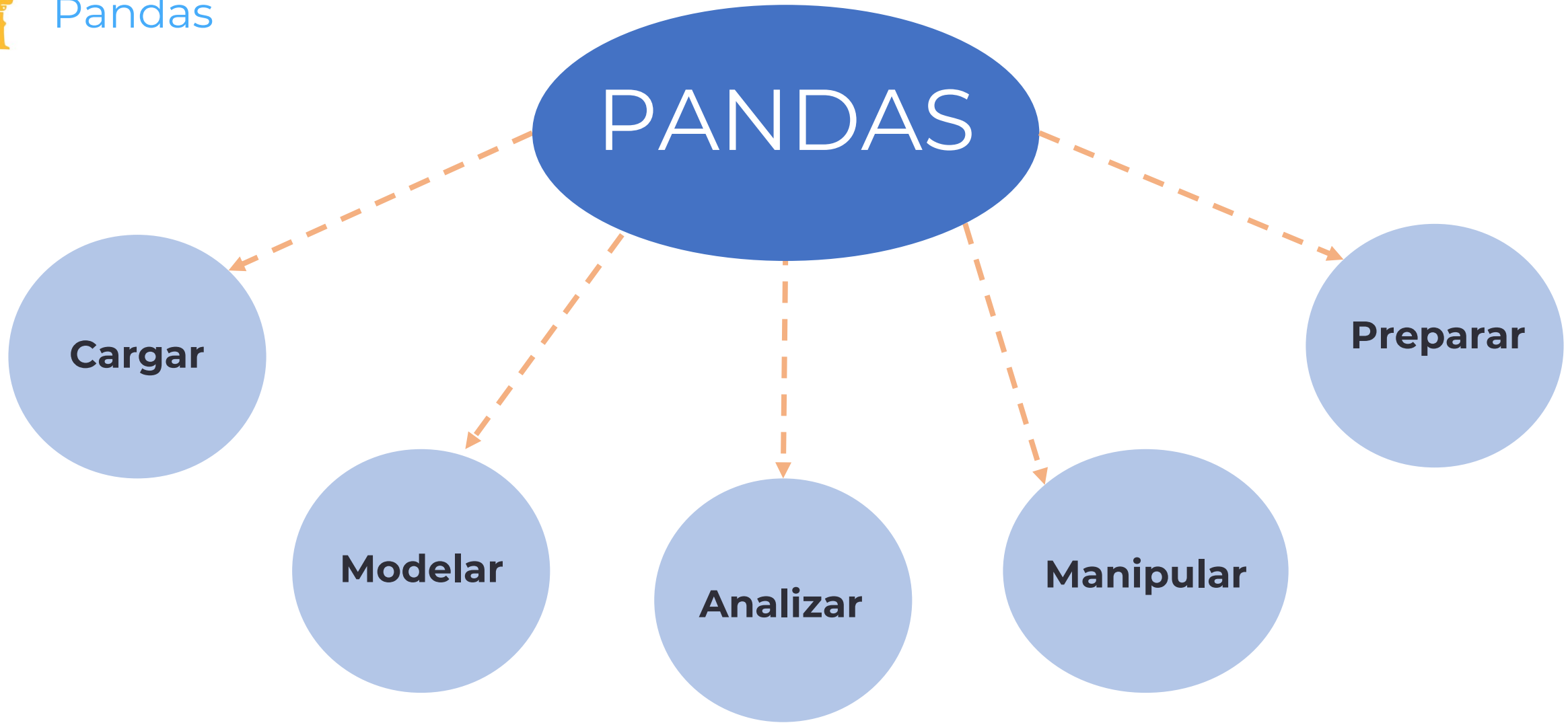
seaborn

Librerías personalizadas

Necesito importarlas

Cada uno puede armar librerías

Pandas



Python

Pandas - Componentes

Pandas

Series

	Ventas
0	$ 230
1	$ 300
2	$ 150
3	$ 200

+

	Productos
0	3
1	5
2	1
3	2

=

DataFrames

	Ventas	Productos
0	$ 230	3
1	$ 300	5
2	$ 150	1
3	$ 200	2

Una Serie es un arreglo unidimensional. Se puede acceder a los datos a través de su posición o índice, similar a un diccionario (son esencialmente una columna)

Un DataFrame es una tabla multidimensional compuesta de una colección de Series. (similares a las tablas de un Excel)

DataFrame

Un dataframe te permite almacenar y manipular datos en una tabla donde las filas son medidas o valores y, cada columna contiene datos para una variable específica.

Consta de 3 componentes:
Datos
Filas
Columnas

Columnas

Filas

0	Nombre	Región	Ventas	Gastos
1	Romina	Norte	31080	22120
2	Juan	Este	42025	17320
3	Claudia	Central	55200	20193
4	Matías	Norte	28010	11000
5	Micaela	Sur	36002	12091
6	Tomás	Oeste	41500	25231
7	Melina	Sur	40153	10210
8	Elías	Central	53110	16530

Datos

DataFrame

Pandas DataFrame es una herramienta de manipulación de datos de alto nivel

Crear un DataFrame
Un DataFrame se puede crear
a partir de listas, diccionarios, archivos, etc.

<u>Ejemplos:</u>

```
ciudades = ['Buenos Aires', 'Córdoba', 'Salta', 'Tucumán', 'Chubut']

df = pd.DataFrame(ciudades)
df = pd.read_csv("ciudades.csv", index_col = 0)
df = pd.read_excel("ciudades.xlsx")
```

Lista

csv

excel

Universidad de SanAndrés

DataFrame

Pandas DataFrame es una herramienta de manipulación de datos de alto nivel

Guardar los datos

Los datos se pueden guardar en diferentes formatos: excel, csv, txt, o en bases de datos.

Ejemplos:

```
df.to_csv("ventas.csv")

df.to_excel(excel_writer="ventas.xlsx", sheet_name="2020",
        header=True, index=False, engine="openpyxl")
```

csv

excel

excel_writer: ruta y nombre del archivo
sheet_name: str. Nombre de la hoja que contendrá al DataFrame (por defecto 'Sheet1')
header: bool o lista de str. Escribir el nombres de las columnas (predeterminado True)
index: bool. Escribir el nombre de las filas (índice) (predeterminado True)
engine: str, Detallar el motor que se va a utilizar: 'openpyxl' o 'xlsxwriter' (opcional)

Parámetros

Python

Pandas – Dataframe – (3/11)

DataFrame

Métodos

Método	Descripción
head()	Muestra las primeras cinco filas de su DataFrame de forma predeterminada
tail()	Muestra las últimas cinco filas de su DataFrame de forma predeterminada
info()	Proporciona los detalles esenciales sobre su conjunto de datos, como el número de filas y columnas, el número de valores no nulos, qué tipo de datos hay en cada columna y cuánta memoria está usando su DataFrame
describe()	Describe los datos
copy()	Hace una copia de los índices y datos del DataFrame
drop_duplicates()	Devuelve una copia de su DataFrame, pero con los duplicados eliminados
columns	Devuelve los nombres de columna de nuestro conjunto de datos
rename()	Renombra ciertas o todas las columnas
dtypes	Esto devuelve una serie con el tipo de datos de cada columna.
shape	Genera una tupla de formato: (cantidad filas, cantidad columnas)
isnull()	Devuelve un DataFrame donde cada celda es Verdadero o Falso dependiendo del estado nulo de esa celda.
dropna()	Elimina cualquier fila con al menos un único valor nulo. Devolverá un nuevo DataFrame sin alterar el original.

DataFrame – Manipulación – Filas y columnas

df.iloc[:, 0] **df.iloc[:, -1]**

	Nombre	Región	Ventas	Gastos
0	Romina	Norte	31080	22120
1	Juan	Este	42025	17320
2	Claudia	Central	55200	20193
3	Matías	Norte	28010	11000
4	Micaela	Sur	36002	12091
5	Tomás	Oeste	41500	25231
6	Melina	Sur	40153	10210
7	Elías	Central	53110	16530

df.iloc[0]

df.iloc[2:5,]

df.iloc[-1]

Nota: usando valores negativos podemos acceder a la última posición sin necesidad de conocer el tamaño del conjunto de datos.

DataFrame – Manipulación – Filas y columnas

df.loc[:, 'Nombre']

	Nombre	Región	Ventas	Gastos
0	Romina	Norte	31080	22120
1	Juan	Este	42025	17320
2	Claudia	Central	55200	20193
3	Matías	Norte	28010	11000
4	Micaela	Sur	36002	12091
5	Tomás	Oeste	41500	25231
6	Melina	Sur	40153	10210
7	Elías	Central	53110	16530

df.loc[:, ['Nombre', 'Región']]

df[['Gastos']]

.loc[]: permite seleccionar filas o columnas en base a

- una **etiqueta** o,
- una **condición**

df.loc[df.loc[:, 'Región'] == 'Sur']

	Nombre	Región	Ventas	Gastos
4	Micaela	Sur	36002	12091
6	Melina	Sur	40153	10210

Nota: también podemos acceder a las columnas llamándolas por su nombre

DataFrame – Manipulación – Filas y columnas

df.dtypes: se utiliza para saber el tipo de dato que almacena la columna

df.dtypes

comando - ejemplo

```
Nombre    object
Región    object
Ventas     int64
Gastos     int64
Dtype: object
```

astype(): se utiliza para convertir alguno de los tipos de datos tabulados del dataframe

df["Región"].astype('category')

comando - ejemplo

```
0      Norte
1       Este
2    Central
3      Norte
4        Sur
5      Oeste
6        Sur
7    Central
Name: Región, dtype: category
Categories (5, object): [Central, Este,
Norte, Oeste, Sur]
```

Universidad de San Andrés

DataFrame – Manipulación – Agregar Datos

Las mas utilizadas son:
1. Crear un listado de los valores a agregar y asignarlos a un dataframe como si fuese un diccionario
2. Con el método insert(...). Con el cual tenemos mayor control de como agregar los datos

año = [2020, 2020, 2019, 2020, 2019, 2019, 2020,2019]

comando - ejemplo

```
# Método 1
df["Año"]=año
```

comando - ejemplo

```
# Método 2 - insert
df.insert(1, "Año", año, True)
```

	Nombre	Región	Ventas	Gastos	Año
0	Romina	Norte	31080	22120	2020
1	Juan	Este	42025	17320	2020
2	Claudia	Central	55200	20193	2019
3	Matías	Norte	28010	11000	2020
4	Micaela	Sur	36002	12091	2019
5	Tomás	Oeste	41500	25231	2019
6	Melina	Sur	40153	10210	2020
7	Elías	Central	53110	16530	2019

DataFrame – Manipulación – Agregar Datos

Merge: agregar datos desde otro dataframe

df3 = df1.**merge(**df2, left_index=True, right_index=True**)** # unir a través de sus índices
df1.**merge(**df2, left_on='Nombre', right_on='Nombre'**)** # unir a través de sus columnas

	Nombre	Año	Región	Ventas	Gastos
0	Romina	2020	Norte	31080	22120
1	Juan	2020	Este	42025	17320
2	Claudia	2019	Central	55200	20193
3	Matías	2020	Norte	28010	11000
4	Micaela	2019	Sur	36002	12091
5	Tomás	2019	Oeste	41500	25231
6	Melina	2020	Sur	40153	10210
7	Elías	2019	Central	53110	16530

df1

+

	Vta+Envío
0	31433
1	42107
2	55293
3	28434
4	36489
5	41937
6	40217
7	53580

df2

=

	Nombre	Año	Región	Ventas	Gastos	Vta+Envío
0	Romina	2020	Norte	31080	22120	31433
1	Juan	2020	Este	42025	17320	42107
2	Claudia	2019	Central	55200	20193	55293
3	Matías	2020	Norte	28010	11000	28434
4	Micaela	2019	Sur	36002	12091	36489
5	Tomás	2019	Oeste	41500	25231	41937
6	Melina	2020	Sur	40153	10210	40217
7	Elías	2019	Central	53110	16530	53580

df3

Universidad de SanAndrés

DataFrame – Manipulación – Ordenar Datos

Los datos se pueden ordenar a través de sus columnas
sort_values([lista de columnas])

inplace=True modificará el objeto DataFrame en lugar de hacer una copia del mismo.

df.**sort_values**(**by**=["Año", "Ventas"], **inplace**=True)

	Nombre	Año	Región	Ventas	Gastos	Vta+Envío
4	Micaela	2019	Sur	36002	12091	36489
5	Tomás	2019	Oeste	41500	25231	41937
7	Elías	2019	Central	53110	16530	53580
2	Claudia	2019	Central	55200	20193	55293
3	Matías	2020	Norte	28010	11000	28434
0	Romina	2020	Norte	31080	22120	31433
6	Melina	2020	Sur	40153	10210	40217
1	Juan	2020	Este	42025	17320	42107

df.**sort_index**(**inplace**=True, **ascending**=True)

	Nombre	Año	Región	Ventas	Gastos	Vta+Envío
0	Romina	2020	Norte	31080	22120	31433
1	Juan	2020	Este	42025	17320	42107
2	Claudia	2019	Central	55200	20193	55293
3	Matías	2020	Norte	28010	11000	28434
4	Micaela	2019	Sur	36002	12091	36489
5	Tomás	2019	Oeste	41500	25231	41937
6	Melina	2020	Sur	40153	10210	40217
7	Elías	2019	Central	53110	16530	53580

Nota: Por defecto se ordenan los valores de forma ascendente, para ordenarlos de forma descendiente, debemos pasar el parámetro ascending=False

DataFrame – Manipulación – Borrar Datos

Para borrar una columna o filas **drop([columnas,], axis=,...)**

Con axis igual a:

- 0: borrar filas
- 1: borrar columnas

	Nombre	Año	Región	Ventas	Gastos	Vta+Envío
0	Romina	2020	Norte	31080	22120	31439
1	Juan	2020	Este	42025	17320	42107
2	Claudia	2019	Central	55200	20193	55605
3	Matías	2020	Norte	28010	11000	28063
4	Micaela	2019	Sur	36002	12091	36166
5	Tomás	2019	Oeste	41500	25231	41671
6	Melina	2020	Sur	40153	10210	40201

df.**drop**(['Año'], **axis=1**)

df.**drop**([1, 5], **axis**=0)

	Nombre	Región	Ventas	Gastos	Vta+Envío
0	Romina	Norte	31080	22120	31439
1	Juan	Este	42025	17320	42107
2	Claudia	Central	55200	20193	55605
3	Matías	Norte	28010	11000	28063
4	Micaela	Sur	36002	12091	36166
5	Tomás	Oeste	41500	25231	41671
6	Melina	Sur	40153	10210	40201

	Nombre	Año	Región	Ventas	Gastos	Vta+Envío
0	Romina	2020	Norte	31080	22120	31439
2	Claudia	2019	Central	55200	20193	55605
3	Matías	2020	Norte	28010	11000	28063
4	Micaela	2019	Sur	36002	12091	36166
6	Melina	2020	Sur	40153	10210	40201

DataFrame – Manipulación – Pivotear

Utilizando la función **pivot()** se puede crear una nueva tabla a partir de la original

Se pueden pasar los siguientes argumentos:

- **values:** qué valores del DataFrame original deseamos utilizar para generar esta nueva tabla dinámica
- **columns:** cuales son las columnas con las cuales se desea conformar la tabla resultante
- **index:** es el índice de la tabla
- **aggfunc:** lista de funciones que se aplicarán a cada valor en forma de agregación

df.pivot_table(**index=**['Región', 'Año'], **values=**['Ventas'])

		Ventas
Región	**Año**	
Central	**2019**	54155
Este	**2020**	42025
Norte	**2020**	29545
Oeste	**2019**	41500
Sur	**2019**	36002
	2020	40153

Tarea en Clase

Combustibles

- Abrir un colab, cargar el dataset de combustibles y guardarlo en un dataframe
- Imprimir las primeras 5 filas del dataframe creado a partir del archivo de combustibles
- Imprimir los nombres de las columnas del dataframe y el tipo de datos de cada columna
- Eliminar una columna y guardar el resultado en un nuevo dataframe
- Armar un subset con las filas 5 a 20 y las columnas 2 y 3, guardarlo en un nuevo dataframe, imprimirlo en pantalla